SC Assignment 2

Liam Erasmus

2026-03-03

Table of contents

Statistical Computing Assignment 2

Parallel Computing Practical

1 Parallel Computing

1.1 Parallel Computing Prac

1.1.1 Question 1

```
library(foreach)
library(kableExtra)

one <- as.data.frame( foreach(i = 1:100, .combine = rbind) %do%{
  samp <- rexp(100, 1)
  c(
  x_bar <- mean(samp),
  vari <- var(samp))
  #results = rbind(x_bar, vari)
}
)
kable(one,
      col.names = c("Sample Mean", "Sample Variance"))
```

	Sample Mean	Sample Variance
result.1	0.9495486	0.6725643
result.2	1.1154272	1.1217824
result.3	1.0217280	1.0648863
result.4	0.9302311	0.8031639
result.5	1.0292952	1.0106472
result.6	1.0142834	0.8473286
result.7	1.0077777	0.7517721
result.8	0.8903922	0.6050788
result.9	0.9751467	0.9006963
result.10	0.9918744	0.8375883
result.11	1.0961606	1.0081425
result.12	1.0064086	0.7160795
result.13	1.0278403	0.9622182

	Sample Mean	Sample Variance
result.14	1.1623981	1.1416654
result.15	1.1650671	1.4182945
result.16	1.0039318	0.9682283
result.17	0.9112110	0.8358095
result.18	1.0956739	1.3903387
result.19	1.0774842	1.0236656
result.20	0.9280967	0.9421134
result.21	0.9344016	0.8822905
result.22	0.9503011	0.8823313
result.23	0.9176143	1.0039944
result.24	0.8147968	0.7024149
result.25	1.2059962	1.3261687
result.26	1.0091492	0.9834171
result.27	0.9259371	0.8436214
result.28	0.8674897	0.7613104
result.29	0.9064548	0.9938133
result.30	1.0233771	0.7941209
result.31	0.9687878	0.8254145
result.32	0.9316200	0.8420258
result.33	1.0404563	0.9957306
result.34	1.0734633	1.4292596
result.35	1.0406984	0.9519933
result.36	0.9536487	0.7614973
result.37	1.0328533	1.1934858
result.38	1.0491855	1.1037017
result.39	1.1291530	1.4061316
result.40	0.8614985	0.8438528
result.41	0.8731938	0.5264359
result.42	1.0658161	0.9754243
result.43	1.0131068	1.0689437
result.44	0.9589133	0.8974351
result.45	0.8360906	0.7876532
result.46	1.1054644	0.9769669
result.47	0.9866849	1.0804131
result.48	1.1011844	1.0143395
result.49	1.1254445	1.0006091
result.50	0.9489763	0.8767032
result.51	1.1211738	1.1974728
result.52	1.1187822	1.7865175
result.53	1.0671117	1.1035389
result.54	0.9678575	0.9934384

	Sample Mean	Sample Variance
result.55	1.0199134	1.1808073
result.56	1.0197670	0.7464509
result.57	0.9578112	0.7019851
result.58	0.9999834	0.9047992
result.59	1.0094579	0.6912868
result.60	0.8982506	0.8476977
result.61	1.1339916	1.0665465
result.62	0.9812783	0.9538758
result.63	0.8776328	0.9804705
result.64	1.0706546	1.1942735
result.65	0.9483170	0.5653396
result.66	0.9100855	1.1414858
result.67	1.1062779	1.3227938
result.68	1.0661938	1.1828913
result.69	1.1421681	1.2788817
result.70	1.0616112	1.2362994
result.71	1.1785199	1.4355900
result.72	1.0812375	1.0502369
result.73	0.8289460	0.8149409
result.74	0.9545333	0.7437740
result.75	1.1314192	0.9744446
result.76	0.9304369	0.9638075
result.77	0.8549258	0.5875020
result.78	1.1349146	1.4157314
result.79	0.9690955	1.4159527
result.80	0.9655151	0.9215751
result.81	0.9578714	0.7321513
result.82	1.0456958	0.6428399
result.83	1.0248847	0.9487222
result.84	1.1484969	2.5347222
result.85	1.0101995	1.1322006
result.86	0.9659579	0.7974103
result.87	0.8363902	0.6165866
result.88	0.8731462	0.9878691
result.89	0.9789216	0.8260660
result.90	0.9657940	0.7258572
result.91	1.0376791	1.2181888
result.92	0.8912083	0.8268415
result.93	1.3749568	1.7959121
result.94	0.7920508	0.5944921
result.95	0.7192900	0.3967262

	Sample Mean	Sample Variance
result.96	0.9986188	1.0612132
result.97	0.9580742	0.8895228
result.98	0.9837503	1.2345943
result.99	1.0959657	1.5522814
result.100	0.9759613	0.6701394

1.1.2 Question 2

```
library(doParallel)
```

```
Loading required package: iterators
```

```
Loading required package: parallel
```

```
library(MASS)

x <- galaxies
n <- length(x)
B <- 200000

# Serial Bootstrap
t_serial <- system.time({
  med_serial <- replicate(B, median(sample(x, n, replace = TRUE)))
})["elapsed"]

t_serial
```

```
elapsed
   2.52
```

```
# One Bootstrap per task (in parallel)

cl <- makeCluster(detectCores() - 1) # do not want my computer to crashhhhhh
registerDoParallel(cl)

t_par_single <- system.time({
  med_par_single <- foreach(i = 1:B, .combine = c) %dopar% {
```

```
    median(sample(x, n, replace = TRUE))
  }
})["elapsed"]

stopCluster(cl)
t_par_single #MUCH SLOWER
```

```
elapsed
 14.419
```

```
# 1000 Bootstraps per task
chunk_size <- 1000
nchunks <- 200

cl <- makeCluster(detectCores() - 1)
registerDoParallel(cl)

t_par_chunk <- system.time({
  med_par_chunk <- foreach(k = 1:nchunks, .combine = c) %dopar% {
    m <- if (k < nchunks) chunk_size else (B - (nchunks - 1) * chunk_size)
    replicate(m, median(sample(x, n, replace = TRUE)))
  }
})["elapsed"]

stopCluster(cl)

t_par_chunk # MUUUUCH FASTER
```

```
elapsed
  0.482
```

1.1.3 Question 3

```
n <- 50
mu_true <- 1
B <- 2000
M <- 10000

cl3 <- makeCluster(detectCores() - 1)
```

```r
registerDoParallel(cl)

coverage_vec <- foreach(m = 1:M, .combine = c) %dopar% {

  x <- rexp(n, rate = 1)

  boot_means <- replicate(B, mean(sample(x, n, replace = TRUE)))

  ci <- quantile(boot_means, probs = c(0.025, 0.975), names = FALSE)

  (ci[1] <= mu_true) && (mu_true <= ci[2])
}

stopCluster(cl3)

mean(coverage_vec)
```

```
[1] 0.9299
```

1.1.4 Question 4

```r
library(iterators)

set.seed(1234)

foreach(x = irnorm(3, count = 5), .combine = c) %do% {
  max(x)
}
```

```
[1]  1.0844412  0.5060559 -0.5466319 -0.4771927  0.9594941
```

1.1.5 Question 5

```r
library(parallel)

R <- 50000
set.seed(1234)
```

```r
t_rep <- system.time(replicate
                     (R, replicate
                       (3, max(rnorm(5))))["elapsed"]

set.seed(1234)
t_for <- system.time(replicate(R,
  foreach(x = irnorm(3, count = 5), .combine = c) %do% max(x)
))["elapsed"]

cl <- makeCluster(detectCores() - 1)
clusterSetRNGStream(cl, 1234)
t_par <- system.time(replicate(R,
  unlist(parLapply(cl, 1:3, function(i) max(rnorm(5))))
))["elapsed"]
stopCluster(cl)

c(replicate = t_rep, foreach = t_for, parLapply = t_par)
```

```
replicate.elapsed   foreach.elapsed parLapply.elapsed
            0.367            24.152            15.218
```